TFG: Despliegue de Infraestructura de Red Segura Híbrida en AWS con Monitorización Centralizada

Autores: "Víctor Alberjón Hidalgo y Pablo Arias Montilla"

---

1. Planificación inicial

En esta fase se establecen las bases del proyecto, definiendo el alcance, los recursos necesarios y las restricciones que guiarán el desarrollo.

1.1. Objetivo

El objetivo principal es diseñar e implantar una infraestructura de red segura en la nube pública (AWS), utilizando una arquitectura de Gateway Linux (Ubuntu) para proteger y enrutar el tráfico de una subred privada donde reside un servidor Windows. Adicionalmente, se busca implementar un sistema de observabilidad que centralice la monitorización de tráfico y recursos de ambos servidores, integrando mecanismos de alerta temprana ante fallos críticos, todo ello bajo un paradigma de **automatización y GitOps**.

1.2. Alcance del proyecto

El proyecto contempla la ejecución de las siguientes tareas clave:

- **Infraestructura como Código (IaC):** Automatización total del despliegue mediante AWS CloudFormation (GitOps).
- **Despliegue de Red (VPC):** Configuración de una nube privada virtual segmentada en subredes pública (DMZ) y privada (Intranet).
- **Servidor de Borde (Ubuntu Server):** Configuración como NAT Instance, seguridad perimetral (iptables) y despliegue de stack de monitorización (Prometheus + Grafana).
- **Servidor Interno (Windows Server):** Aislamiento de red (sin IP pública directa) y configuración de servicios internos (Active Directory).
- **Active Directory:** Despliegue de dominio Windows para gestión centralizada de identidades.
- **VPN (WireGuard):** Configuración de túnel VPN para acceso remoto seguro de usuarios al dominio.
- **Gestión y Acceso:** Acceso remoto seguro mediante SSH y RDP (vía Port Forwarding en el Gateway).
- **Interconexión y Visibilidad:** Configuración de tablas de rutas para forzar el tráfico a través del Gateway.

1.3. Recursos identificados

- **Infraestructura (AWS EC2 & CloudFormation):**
  - 1x Instancia **t3.micro** (Ubuntu Server 22.04 LTS) + **IP Elástica**.
  - 1x Instancia **t3.small** (Windows Server 2022). *Nota: Se aumentó de micro a small para garantizar la estabilidad del sistema.*
- **Software y Servicios:**
  - Red y Automatización: IPTables, VPC Routing, YAML (CloudFormation).
  - Monitorización: Prometheus, Grafana, Alertmanager, Node Exporter, Windows Exporter.
  - VPN: WireGuard para acceso remoto seguro.

1.4. Restricciones y condicionantes

- **Económicas:** Viabilidad dentro de la capa gratuita (Free Tier) de AWS siempre que sea posible.
- **Seguridad:** Administración a través de puertos estándar (22 y 3389) protegidos por Security Groups.
- **Plazos:** Despliegue funcional antes de la fecha de defensa del TFG.

1.5. Cronograma preliminar

Fase	Tarea Principal	Semanas	Descripción
Fase 1	Planificación	1 - 2	Definición de alcance y estudio de viabilidad.
Fase 2	Diseño	3 - 4	Diagramas de red VPC y políticas de seguridad.
Fase 3	Despliegue Infra	5 - 6	Creación de VPC, subredes y automatización IaC.
Fase 4	Configuración	7 - 8	Configuración de NAT, Firewall y Monitorización.
Fase 5	Accesos	9	Habilitación de SSH y RDP mediante Port Forwarding.
Fase 6	Validación	10	Pruebas de conectividad y estrés.

Fase	Tarea Principal	Semanas	Descripción
Fase 7	Documentación	11 - 12	Redacción final de la memoria y defensa.

---

2. Análisis de requisitos

2.1. Requisitos Funcionales (RF)

- **RF-01 (Enrutamiento NAT):** El servidor Ubuntu debe actuar como puerta de enlace para la subred privada.
- **RF-02 (Despliegue Automatizado):** La infraestructura debe poder recrearse de forma idéntica sin intervención manual.
- **RF-03 (Seguridad de Red):** Gestión del tráfico mediante firewall permitiendo solo administración autorizada.
- **RF-04 (Monitorización de Tráfico):** Captura de métricas de ancho de banda en la interfaz del Gateway.
- **RF-05 (Dashboard Unificado):** Grafana debe mostrar el estado de recursos de ambos servidores.
- **RF-06 (Active Directory):** Implementar dominio Windows con AD para gestión centralizada de usuarios y recursos.
- **RF-07 (Acceso VPN):** Configurar WireGuard para permitir acceso remoto seguro de usuarios al dominio corporativo.
- **RF-08 (Gestión de Alertas):** Notificaciones en tiempo real (Telegram/Discord) ante anomalías críticas.
- **RF-09 (Acceso Remoto):** Garantizar acceso administrativo vía SSH (22) y RDP (3389).

2.2. Requisitos No Funcionales (RNF)

- **RNF-01 (Aislamiento):** La instancia Windows debe residir en una subred privada sin IP pública directa.
- **RNF-02 (Inmutabilidad):** Aplicación de principios GitOps para garantizar la consistencia del entorno.
- **RNF-03 (Disponibilidad):** Los servicios de enrutamiento deben iniciarse automáticamente (systemd).
- **RNF-04 (Seguridad VPN):** Las conexiones VPN deben utilizar cifrado fuerte. WireGuard emplea criptografía moderna (ChaCha20, Curve25519).
- **RNF-05 (Rendimiento):** El Gateway debe mantener latencias de enrutamiento aceptables sin afectar significativamente el rendimiento de la red.
- **RNF-06 (Documentación):** Toda configuración debe estar documentada y versionada para garantizar replicabilidad del entorno.

3. Diseño de la solución

3.1. Arquitectura de Red y Topología (AWS VPC)

- **VPC CIDR:** `10.0.0.0/16`
- **Subred Pública (DMZ):** `10.0.1.0/24`. Aloja el Gateway Ubuntu con IP Elástica.
- **Subred Privada (Intranet):** `10.0.2.0/24`. Aloja el Windows Server. Su tráfico `0.0.0.0/0` apunta a la interfaz de red del Ubuntu.

3.2. Diseño de Seguridad y Accesos

1. **SG-Gateway (Ubuntu):** Inbound: 22 (SSH), 3389 (RDP Forward), 3000 (Grafana). Outbound: Todo permitido.
2. **SG-Internal (Windows):** Inbound: 3389 (RDP) e ICMP (Ping) solo desde el SG-Gateway. Outbound: Todo permitido hacia el Gateway.

3.3 Esquema de la arquitectura

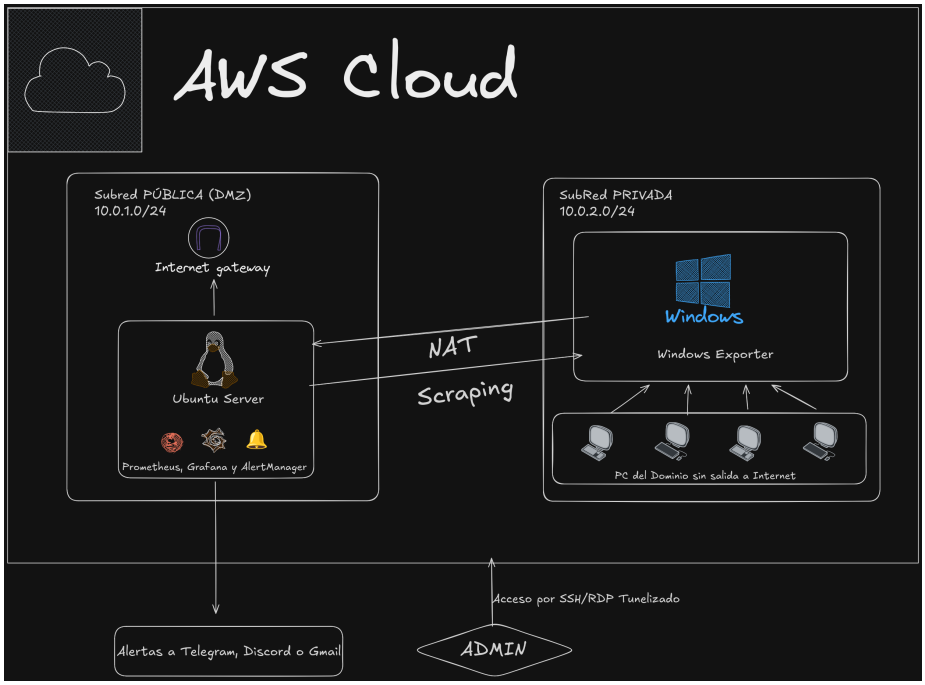

Figura 1: Topología de red en AWS con segmentación de subredes. —

4. Implantación y configuración

4.1. Despliegue de Infraestructura Base en AWS

1. **IP Elástica:** Asociada al Ubuntu para mantener un punto de acceso administrativo fijo.
2. **IP Estática Windows:** Configurada como `10.0.2.75` para evitar desincronización de servicios internos.
3. **Source/Destination Check:** Desactivado en la instancia Ubuntu para permitir el funcionamiento del NAT.

4.2. Configuración del Enrutamiento y NAT en Linux

```
# Habilitar IP Forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Configuración de NAT (Masquerade)
sudo iptables -t nat -A POSTROUTING -s 10.0.2.0/24 -o ens5 -j MASQUERADE

# Reenvío de puerto RDP (DNAT)
sudo iptables -t nat -A PREROUTING -p tcp --dport 3389 -j DNAT --to-destination 10.0.2.75:33

# Persistencia de reglas
sudo apt update && sudo apt install iptables-persistent -y
sudo netfilter-persistent save
```

4.3. Configuración del Servidor Interno (Windows Server)

Para garantizar la estabilidad operativa y la conectividad a través del Gateway, se han realizado las siguientes configuraciones críticas:

1. **Optimización de Instancia:** Migración del servidor a una instancia de familia `t3.small` (2 vCPU, 2GB RAM).
2. **Configuración de Red Estática:** Direccionamiento IPv4 fijado en `10.0.2.75`, Gateway en `10.0.2.1` y DNS apuntando a `127.0.0.1` y `8.8.8.8`.
3. **Gestión del Firewall:** Desactivación del cortafuegos de Windows mediante PowerShell:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled False
```

4.4. Implementación del Controlador de Dominio (Active Directory)

Tras completar la configuración de la infraestructura de red y verificar la conectividad entre las instancias de la VPC, se procedió a la configuración del servidor interno basado en **Windows Server 2022** como controlador de dominio mediante la instalación del rol **Active Directory Domain Services**. El objetivo de esta fase fue establecer un sistema centralizado de autenticación y gestión de identidades para la red privada del proyecto.

Instalación del rol Active Directory La instalación del servicio se realizó a través del **Server Manager**, utilizando el asistente de instalación de roles y características. Se seleccionó el rol **Active Directory Domain Services (AD DS)**, incluyendo automáticamente las dependencias necesarias, entre ellas el servicio **DNS**, fundamental para localizar los distintos servicios del dominio dentro de la red interna.

Una vez completada la instalación del rol, el sistema solicitó **promover el servidor a controlador de dominio**.

Promoción del servidor a controlador de dominio El servidor fue configurado como el **primer controlador de dominio** dentro de un nuevo bosque (*Forest*). Se definieron los siguientes parámetros:

Parámetro	Valor
Dominio del proyecto	`tfg.vp`
Nombre NetBIOS	`TFG`

Durante el proceso de configuración se habilitaron: - Servidor **DNS integrado** - **Catálogo global (Global Catalog)** - Base de datos del directorio de **Active Directory**

Asimismo, se estableció una **contraseña de recuperación para el modo Directory Services Restore Mode (DSRM)**, utilizada para tareas de mantenimiento o recuperación del servicio en caso de fallo crítico.

Tras la validación de requisitos, el sistema instaló los componentes necesarios y **reinició automáticamente el servidor** para finalizar el proceso.

Configuración del servicio DNS Durante la promoción del servidor a controlador de dominio se instaló automáticamente el servicio **DNS integrado**. Este servicio es esencial para el funcionamiento de Active Directory, ya que permite a los equipos de la red localizar: - Controladores de dominio - Servidores LDAP - Otros recursos del directorio

Durante la instalación se mostró una advertencia relacionada con la **imposibilidad de crear una delegación DNS**. Este mensaje es habitual cuando se crea un dominio completamente nuevo dentro de una red privada sin una zona DNS superior existente, por lo que **no afecta al funcionamiento del entorno desplegado**.

Verificación del servicio Una vez completada la instalación, se verificó el correcto funcionamiento del dominio mediante las herramientas administrativas incluidas, especialmente la consola **Active Directory Users and Computers**. Se comprobó: - La creación correcta del dominio - La disponibilidad del

controlador de dominio - El funcionamiento del servicio de autenticación - La correcta resolución de nombres mediante **DNS** dentro de la red privada

Finalmente, se confirmó que el servidor Windows mantiene conectividad hacia Internet a través del servidor Ubuntu configurado como gateway NAT, validando la integración entre la infraestructura de red y los servicios de directorio implementados.

4.5. Implementación de Windows Exporter para Monitorización

Con el objetivo de permitir la monitorización del servidor Windows dentro de la infraestructura del proyecto, se implementó **Windows Exporter**, una herramienta que expone métricas del sistema operativo Windows en un formato compatible con sistemas de monitorización como Prometheus.

Windows Exporter recopila información del sistema como el uso de CPU, memoria, disco y red, y la expone a través de un endpoint HTTP interno que puede ser consultado desde la red privada.

Instalación de Windows Exporter La instalación se realizó en el servidor **Windows Server 2022** mediante el paquete instalador en formato `.msi`. Durante el proceso se configuraron los *collectors*, responsables de recopilar diferentes métricas del sistema.

Módulos habilitados:

Módulo	Descripción
`cpu`	Uso del procesador
`memory`	Consumo de memoria RAM
`logical_disk`	Espacio disponible y uso de discos
`net`	Estadísticas de tráfico de red
`os`	Información general del sistema operativo
`system`	Estado general del sistema y tiempo de actividad

Una vez completada la instalación, Windows Exporter se ejecuta automáticamente como un servicio de Windows llamado **windows_exporter**.

Exposición de métricas Windows Exporter expone las métricas a través de un servidor HTTP que escucha en el puerto **9182**. Para verificar el correcto funcionamiento del servicio, se accedió desde el propio servidor a:

`http://localhost:9182/metrics`

Al abrir esta dirección se muestran múltiples métricas del sistema en formato de texto estructurado. Ejemplos de métricas visibles:

```
windows_cpu_time_total
windows_memory_available_bytes
windows_logical_disk_free_bytes
windows_os_info
windows_system_system_up_time
```

Configuración del firewall Para permitir que otros equipos dentro de la red privada puedan acceder a las métricas, se creó una regla en el firewall de Windows que permite conexiones entrantes al puerto 9182:

```
New-NetFirewallRule -DisplayName "windows_exporter" -Direction Inbound -Protocol TCP -LocalP
```

Con esto, el servidor Windows queda preparado para exponer sus métricas del sistema de manera centralizada, lo que permite su monitorización desde Prometheus en el servidor Gateway.

4.6. Automatización y Enfoque GitOps (Infraestructura como Código)

Para garantizar la inmutabilidad y replicabilidad del entorno, se ha adoptado una metodología GitOps utilizando **AWS CloudFormation**.

Toda la infraestructura descrita en los apartados anteriores (VPC, Subredes, Tablas de Rutas, Security Groups e Instancias EC2) está definida de forma declarativa en un único archivo YAML (`despliegue-tfg.yaml`).

Adicionalmente, se ha implementado un aprovisionamiento de cero toques (**Zero-Touch Provisioning**) utilizando la propiedad `UserData`. Esto permite que, en el momento de crear la pila en AWS, las instancias ejecuten automáticamente sus configuraciones internas:

- El **Ubuntu Gateway** ejecuta un script bash en el arranque que habilita el reenvío de paquetes IP (`ip_forward`) y establece las reglas de iptables (NAT y Port Forwarding) haciéndolas persistentes.
- El **Windows Server** ejecuta un script de PowerShell en el arranque que localiza la interfaz de red activa, le asigna la IP estática requerida para el Active Directory, configura los servidores DNS y desactiva los perfiles del cortafuegos.

Esta aproximación elimina la necesidad de intervención manual inicial (SSH/RDP) para la configuración de red y previene errores humanos durante el despliegue del laboratorio.

4.7. Implementación de VPN con WireGuard

Para permitir el acceso remoto seguro de usuarios a la infraestructura desde fuera de AWS, se ha implementado una VPN basada en **WireGuard**. Esta solución permite que los usuarios se conecten a la red privada y accedan a los recursos del dominio Active Directory de forma segura.

Arquitectura de la VPN

Componente	Descripción
Rango VPN	`10.0.3.0/24` - Subred dedicada para clientes VPN
Puerto	UDP 51820 - Puerto estándar de WireGuard
DNS	`10.0.2.75` - Controlador de dominio Active Directory
Gateway	Ubuntu Server actúa como servidor VPN y NAT

La configuración permite que los clientes VPN accedan a: - **Subred privada** (`10.0.2.0/24`): Windows Server, Active Directory y otros recursos internos - **Subred VPN** (`10.0.3.0/24`): Comunicación entre clientes conectados

Instalación del servidor WireGuard La instalación se realiza en el **Ubuntu Gateway**. Los pasos ejecutados fueron:

```
# Instalar WireGuard
sudo apt update && sudo apt install -y wireguard

# Generar claves del servidor
sudo wg genkey | sudo tee /etc/wireguard/privatekey | sudo wg pubkey | sudo tee /etc/wiregua
sudo chmod 600 /etc/wireguard/privatekey
```

Configuración del servidor El archivo de configuración `/etc/wireguard/wg0.conf` define la interfaz VPN:

```
[Interface]
Address = 10.0.3.1/24
ListenPort = 51820
PrivateKey = <clave_privada_servidor>

# Peers (clientes) se añaden dinámicamente con el script de gestión
```

Reglas de firewall para VPN Se han añadido las siguientes reglas iptables para permitir el tráfico VPN y el enrutamiento hacia la subred privada:

```
# Habilitar IP Forwarding (si no estaba habilitado)
sudo sysctl -w net.ipv4.ip_forward=1

# NAT para tráfico VPN hacia Internet y subred privada
sudo iptables -t nat -A POSTROUTING -s 10.0.3.0/24 -o ens5 -j MASQUERADE

# Persistir reglas
sudo netfilter-persistent save
```

Adicionalmente, el **Security Group de AWS** (`SG-Gateway`) debe permitir tráfico entrante en el puerto **UDP 51820** desde las IPs de origen autorizadas.

Inicio del servicio WireGuard

```
# Habilitar e iniciar el servicio
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0

# Verificar estado
sudo wg show
```

Gestión de clientes VPN Se ha desarrollado un script de automatización (`scripts/crear-cliente-vpn.sh`) que facilita la creación de nuevos clientes VPN. Este script:

1. **Genera claves** del cliente (pública y privada)
2. **Asigna una IP automática** dentro del rango `10.0.3.2-254`
3. **Registra el peer** en el servidor WireGuard
4. **Crea el archivo de configuración** listo para importar
5. **Configura el DNS** para apuntar al Active Directory

Uso del script:

```
# En el servidor Ubuntu Gateway
sudo ./crear-cliente-vpn.sh <nombre_cliente> [ip_elastica]

# Ejemplo
sudo ./crear-cliente-vpn.sh usuario1
sudo ./crear-cliente-vpn.sh portatil_juan 54.123.45.67
```

El script genera un archivo `.conf` en `/etc/wireguard/clients/` que debe ser transferido al dispositivo cliente e importado en la aplicación WireGuard.

Configuración de clientes **Archivo de configuración del cliente (ejemplo):**

```
[Interface]
PrivateKey = <clave_privada_cliente>
Address = 10.0.3.2/24
DNS = 10.0.2.75

[Peer]
PublicKey = <clave_publica_servidor>
Endpoint = <IP_ELASTICA_GATEWAY>:51820
AllowedIPs = 10.0.2.0/24, 10.0.3.0/24
PersistentKeepalive = 25
```

Parámetro	Descripción
`Address`	IP asignada al cliente en la VPN
`DNS`	Servidor DNS (Controlador de Dominio AD)

Parámetro	Descripción
`Endpoint`	IP pública del Gateway y puerto WireGuard
`AllowedIPs`	Rutas que se envían a través de la VPN
`PersistentKeepalive`	Mantiene la conexión activa (útil tras NAT)

Integración con Active Directory Una vez conectado a la VPN, el cliente puede unirse al dominio **tfg.vp**:

1. **Verificar conectividad DNS:**

   ```
   nslookup tfg.vp 10.0.2.75
   ```

2. **Unirse al dominio (Windows):**

   - Configuración → Sistema → Acerca de → Unirse a un dominio
   - Introducir dominio: `tfg.vp`
   - Proporcionar credenciales de administrador del dominio

3. **Unirse al dominio (Linux):**

   ```
   sudo realm join tfg.vp -U administrador
   ```

Verificación de la conexión VPN Para verificar clientes conectados desde el servidor:

```
# Ver peers activos
sudo wg show

# Salida esperada:
# interface: wg0
#   public key: (clave_del_servidor)
#   private key: (hidden)
#   listening port: 51820
#
# peer: (clave_publica_cliente)
#   endpoint: (IP_cliente):puerto
#   allowed ips: 10.0.3.2/32
```

Consideraciones de seguridad

- **Cifrado:** WireGuard utiliza criptografía moderna (ChaCha20, Curve25519, BLAKE2s)
- **Autenticación:** Basada en claves públicas/privadas
- **Punto único de entrada:** Todo el tráfico pasa por el Gateway Ubuntu
- **DNS:** Resolución de nombres a través del Active Directory
- **Acceso restringido:** Los clientes VPN solo pueden acceder a las subredes permitidas

4.8. Implementación de Node Exporter para Monitorización de Ubuntu

Para monitorizar el servidor Ubuntu Gateway, se ha instalado **Node Exporter**, una herramienta que expone métricas del sistema operativo Linux en un formato compatible con Prometheus.

Node Exporter recopila información del sistema como el uso de CPU, memoria, disco y red, y la expone a través de un endpoint HTTP en el puerto 9100.

Instalación de Node Exporter La instalación se realizó desde los repositorios de Ubuntu:

```
sudo apt update
sudo apt install -y prometheus-node-exporter
sudo systemctl enable prometheus-node-exporter
sudo systemctl start prometheus-node-exporter
```

Verificación del servicio Para verificar que Node Exporter está funcionando correctamente:

```
sudo systemctl status prometheus-node-exporter
curl http://localhost:9100/metrics
```

El servicio expone métricas en formato de texto que pueden ser consultadas desde `http://localhost:9100/metrics`.

Métricas disponibles

Métrica	Descripción
`node_cpu_seconds_total`	Tiempo de CPU por modo
`node_memory_MemAvailable_bytes`	Memoria disponible
`node_memory_MemTotal_bytes`	Memoria total
`node_filesystem_avail_bytes`	Espacio disponible en disco
`node_filesystem_size_bytes`	Tamaño total del sistema de archivos
`node_network_receive_bytes_total`	Bytes recibidos por red
`node_network_transmit_bytes_total`	Bytes transmitidos por red

4.9. Implementación de Prometheus para Recolección de Métricas

Prometheus es el sistema de monitorización centralizado que recopila métricas de todos los servidores de la infraestructura y las almacena en una base de datos temporal.

Instalación de Prometheus La instalación se realizó desde los repositorios de Ubuntu:

```
sudo apt install -y prometheus
sudo systemctl enable prometheus
sudo systemctl start prometheus
```

Configuración de Targets El archivo de configuración `/etc/prometheus/prometheus.yml` define los objetivos de recolección:

```yaml
scrape_configs:
  - job_name: 'prometheus'
    scrape_interval: 5s
    scrape_timeout: 5s
    static_configs:
      - targets: ['localhost:9090']

  - job_name: node
    static_configs:
      - targets: ['localhost:9100']

  - job_name: 'windows-server'
    static_configs:
      - targets: ['10.0.2.75:9182']
```

Target	Descripción	Puerto
`prometheus`	Métricas de Prometheus	9090
`node`	Métricas de Ubuntu Gateway	9100
`windows-server`	Métricas de Windows Server	9182

Reglas de Alerta Se han configurado las siguientes reglas de alerta en `/etc/prometheus/alert_rules.yml`:

Regla	Descripción	Umbral
`InstanceDown`	Servidor caído	1 minuto sin respuesta
`HighCPU`	Uso de CPU elevado	>80% durante 5 minutos
`HighMemory`	Uso de memoria elevado	>85% durante 5 minutos
`DiskSpaceLow`	Espacio de disco bajo	>85% de uso durante 5 minutos

Verificación de Targets Para verificar que Prometheus está recolectando métricas correctamente:

```
curl http://localhost:9090/api/v1/targets | python3 -m json.tool
```

Cada target debe mostrar `"health": "up"` en su estado.

4.10. Implementación de Grafana para Visualización

Grafana proporciona una interfaz web para visualizar las métricas recolectadas por Prometheus mediante dashboards personalizables.

Instalación de Grafana Se instaló desde el repositorio oficial de Grafana:

```
sudo apt install -y apt-transport-https software-properties-common wget
wget -q -O /usr/share/keyrings/grafana.key https://apt.grafana.com/gpg.key
echo "deb [signed-by=/usr/share/keyrings/grafana.key] https://apt.grafana.com stable main" |
sudo apt update
sudo apt install -y grafana
sudo systemctl enable grafana-server
sudo systemctl start grafana-server
```

Acceso a Grafana

Parámetro	Valor
URL	`http://<IP_ELASTICA_GATEWAY>:3000`
Usuario por defecto	`admin`
Contraseña por defecto	`admin`

Se recomienda cambiar la contraseña en el primer acceso.

Configuración de Data Source Para conectar Grafana con Prometheus:

1. Acceder a **Configuration** → **Data Sources**
2. Añadir nuevo data source tipo **Prometheus**
3. Configurar URL: `http://localhost:9090`
4. Guardar y probar conexión

Dashboard Recomendado Se recomienda importar el dashboard **Node Exporter Full** (ID: 1860) desde Grafana:

1. **Dashboards** → **Import**
2. Introducir ID: `1860`
3. Seleccionar data source: Prometheus
4. Importar

Este dashboard proporciona una vista completa de las métricas del sistema Ubuntu.

4.11. Implementación de Alertmanager para Notificaciones

Alertmanager gestiona las alertas generadas por Prometheus y las envía a través de diferentes canales de notificación.

Instalación de Alertmanager La versión inicial instalada desde repositorios era la 0.23.0, pero se actualizó a la versión 0.28.1 para soportar notificaciones de Telegram nativas:

```
# Descargar versión 0.28.1
cd /tmp
wget https://github.com/prometheus/alertmanager/releases/download/v0.28.1/alertmanager-0.28.
tar xvf alertmanager-0.28.1.linux-amd64.tar.gz

# Detener servicio
sudo systemctl stop prometheus-alertmanager

# Reemplazar binario
sudo cp /tmp/alertmanager-0.28.1.linux-amd64/alertmanager /usr/bin/prometheus-alertmanager

# Crear directorio de datos
sudo mkdir -p /var/lib/prometheus/alertmanager
sudo chown -R prometheus:prometheus /var/lib/prometheus/alertmanager

# Iniciar servicio
sudo systemctl start prometheus-alertmanager
```

Configuración del servicio El archivo `/etc/default/prometheus-alertmanager` debe contener:

```
ARGS="--storage.path=/var/lib/prometheus/alertmanager --config.file=/etc/prometheus/alertman
```

Configuración de Telegram El archivo `/etc/prometheus/alertmanager.yml` configura las notificaciones:

```yaml
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 1h
  receiver: telegram-notifications

receivers:
  - name: telegram-notifications
```

```
    telegram_configs:
      - bot_token: 'TU_BOT_TOKEN'
        chat_id: TU_CHAT_ID
        parse_mode: HTML

inhibit_rules:
  - source_match:
      severity: critical
    target_match:
      severity: warning
    equal: ['alertname', 'instance']
```

Creación del Bot de Telegram Para configurar las notificaciones por Telegram:

1. **Crear un bot:** Abrir @BotFather en Telegram y ejecutar `/newbot`
2. **Obtener el token:** Guardar el token proporcionado (ej: `1234567890:ABCdef...`)
3. **Crear un grupo:** Crear un grupo de Telegram para recibir alertas
4. **Añadir el bot:** Añadir el bot al grupo como administrador
5. **Obtener el Chat ID:** Acceder a `https://api.telegram.org/bot<TOKEN>/getUpdates` y buscar el campo `chat.id`
6. **Configurar en Alertmanager:** Añadir el token y Chat ID en `/etc/prometheus/alertmanager.yml`

Prueba de Alertas Para enviar una alerta de prueba:

```
amtool alert add prueba severity=critical --annotation=summary="Alerta de prueba" --annotati
```

La alerta debe llegar al grupo de Telegram configurado.

Ejemplo de alerta recibida A continuación se muestra un ejemplo real de una alerta recibida en Telegram cuando el servidor Windows deja de responder:

Ejemplo de alerta `InstanceDown` recibida en Telegram indicando que el servidor Windows (10.0.2.75:9182) está caído.

Verificación del Estado Para verificar que Alertmanager está funcionando:

```
sudo systemctl status prometheus-alertmanager
curl http://localhost:9093/api/v2/status | python3 -m json.tool
```

Estado de las Notificaciones

Canal	Estado	Notas
Telegram	Funcional	Configurado con bot token y Chat ID

Canal	Estado	Notas
Discord	Pendiente	Requiere servicio intermedio para formato JSON

Archivos de Configuración Los archivos de configuración de ejemplo están disponibles en el directorio `configs/` del repositorio:

Archivo	Descripción
`configs/prometheus.yml.example`	Configuración de Prometheus
`configs/alert_rules.yml`	Reglas de alerta
`configs/alertmanager.yml.example`	Configuración de Alertmanager
`configs/prometheus-alertmanager.defaults`	Variables de entorno de Alertmanager

---

5. Trabajo pendiente

5.1. Configuración de notificaciones con Discord

Discord requiere un servicio intermedio que transforme el formato de alertas de Alertmanager al formato JSON esperado por Discord (`{"content": "mensaje"}`). Se deja pendiente para futuras implementaciones.

Posibles soluciones: - Crear un servicio web intermedio que reciba webhooks de Alertmanager y los transforme - Utilizar servicios como Prometheus Discord Webhook

5.2. Instalación de Windows Exporter

El Windows Exporter debe instalarse en el servidor Windows (10.0.2.75) para completar la monitorización. La instalación se documenta en la sección 4.5.

Pasos pendientes: 1. Descargar Windows Exporter desde GitHub 2. Instalar con los módulos necesarios: `cpu`, `memory`, `logical_disk`, `net`, `os`, `system` 3. Verificar métricas en `http://10.0.2.75:9182/metrics` 4. Importar dashboard de Windows Server en Grafana

5.3. Dashboard de Grafana para Windows Server

Una vez instalado Windows Exporter, se recomienda importar el dashboard de Windows Server en Grafana. Dashboard recomendado: Windows Node Exporter (ID: 12496).

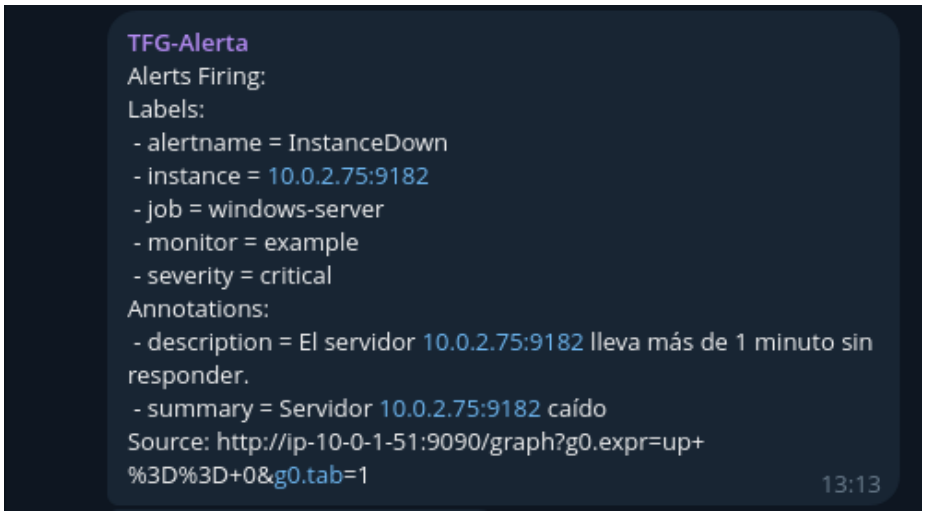

Figure 1: Alerta de Windows Server caído

5.4. Pruebas y Validación

Pendientes de realizar: - Pruebas de conectividad VPN desde clientes externos - Pruebas de unión al dominio Active Directory desde clientes VPN - Pruebas de carga en la infraestructura - Pruebas de failover de servicios